Database: NoSQL (MongoDB)

Section D — AI-Augmented Learning

STEP 1 · BUILD WITH AI

1. Exact Prompt Given to the AI Tool

Write a MongoDB aggregation pipeline for a collection called deliveries. Each document has the fields driverId (string), orderId (string), restaurantName (string), deliveryTimeMinutes (number), tip (number), and status (string).

The pipeline must:

1. Use $match to filter only deliveries where status is "completed".
2. Use $group by driverId.
3. Calculate the average delivery time as avgDeliveryTime.
4. Calculate total tips as totalTips.
5. Use $sort to sort by avgDeliveryTime in ascending order so the fastest drivers appear first.
6. Use $project to rename the output fields clearly as driverID, avgDeliveryTime, and totalTips.
7. Suppress the default _id field.

Return only the MongoDB aggregation pipeline.

2. AI's Original Pipeline

```
db.deliveries.aggregate([
  {
    $match: {
      status: "completed"
    }
  },
  {
    $group: {
      _id: "$driverId",
      avgDeliveryTime: {
        $avg: "$deliveryTimeMinutes"
      },
      totalTips: {
        $sum: "$tip"
      }
    }
  },
```

```
  {
    $sort: {
      avgDeliveryTime: 1
    }
  },
  {
    $project: {
      _id: 0,
      driverID: "$_id",
      avgDeliveryTime: 1,
      totalTips: 1
    }
  }
])
```

STEP 2 · TEST & DEBUG WITHOUT AI

Sample Data Tested in mongosh

```
db.deliveries.insertMany([
  {
    driverId: "D001",
    orderId: "O101",
    restaurantName: "Spice Garden",
    deliveryTimeMinutes: 25,
    tip: 50,
    status: "completed"
  },
  {
    driverId: "D001",
    orderId: "O102",
    restaurantName: "Pizza Palace",
    deliveryTimeMinutes: 35,
    tip: 30,
    status: "completed"
  },
  {
    driverId: "D002",
    orderId: "O103",
    restaurantName: "Wok Express",
    deliveryTimeMinutes: 20,
    tip: 40,
    status: "completed"
  },
  {
    driverId: "D002",
    orderId: "O104",
```

```
    restaurantName: "Burger Hub",
    deliveryTimeMinutes: 30,
    tip: 60,
    status: "completed"
  },
  {
    driverId: "D003",
    orderId: "O105",
    restaurantName: "Green Leaf Cafe",
    deliveryTimeMinutes: 45,
    tip: 20,
    status: "pending"
  }
])
Corrected Version
db.deliveries.aggregate([
  {
    $match: {
      status: "completed"
    }
  },
  {
    $group: {
      _id: "$driverId",
      avgDeliveryTime: {
        $avg: "$deliveryTimeMinutes"
      },
      totalTips: {
        $sum: "$tip"
      }
    }
  },
  {
    $project: {
      _id: 0,
      driverID: "$_id",
      avgDeliveryTime: 1,
      totalTips: 1
    }
  },
  {
    $sort: {
      avgDeliveryTime: 1
    }
  }
])
```

3. Testing / Debugging Note

The AI's original pipeline produced the correct results, but I identified an improvement in the stage order. I moved $project before $sort so that the final output fields are already clearly named before the sorting stage. The pipeline still sorts by avgDeliveryTime ascending, while _id is suppressed in the final output. I also tested the pipeline with a pending delivery to confirm that $match correctly excludes non-completed deliveries.

Expected Result

D002 | avgDeliveryTime: 25 | totalTips: 100

D001 | avgDeliveryTime: 30 | totalTips: 80

The D003 delivery is excluded because its status is "pending".